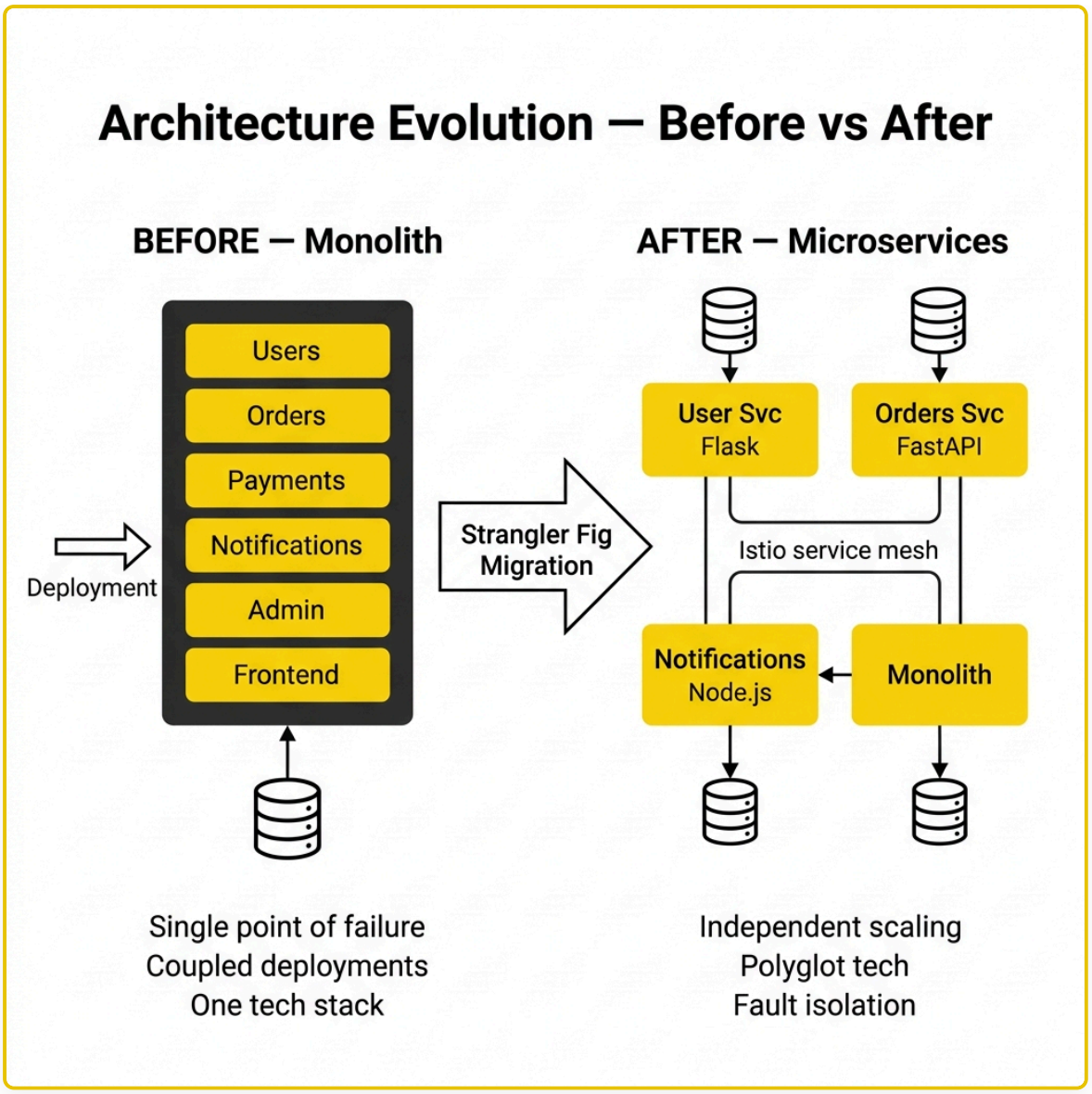
Strangler Fig Migration: Monolith to Microservices

A Zero-Downtime Architectural Migration on AWS EKS with Istio Service Mesh

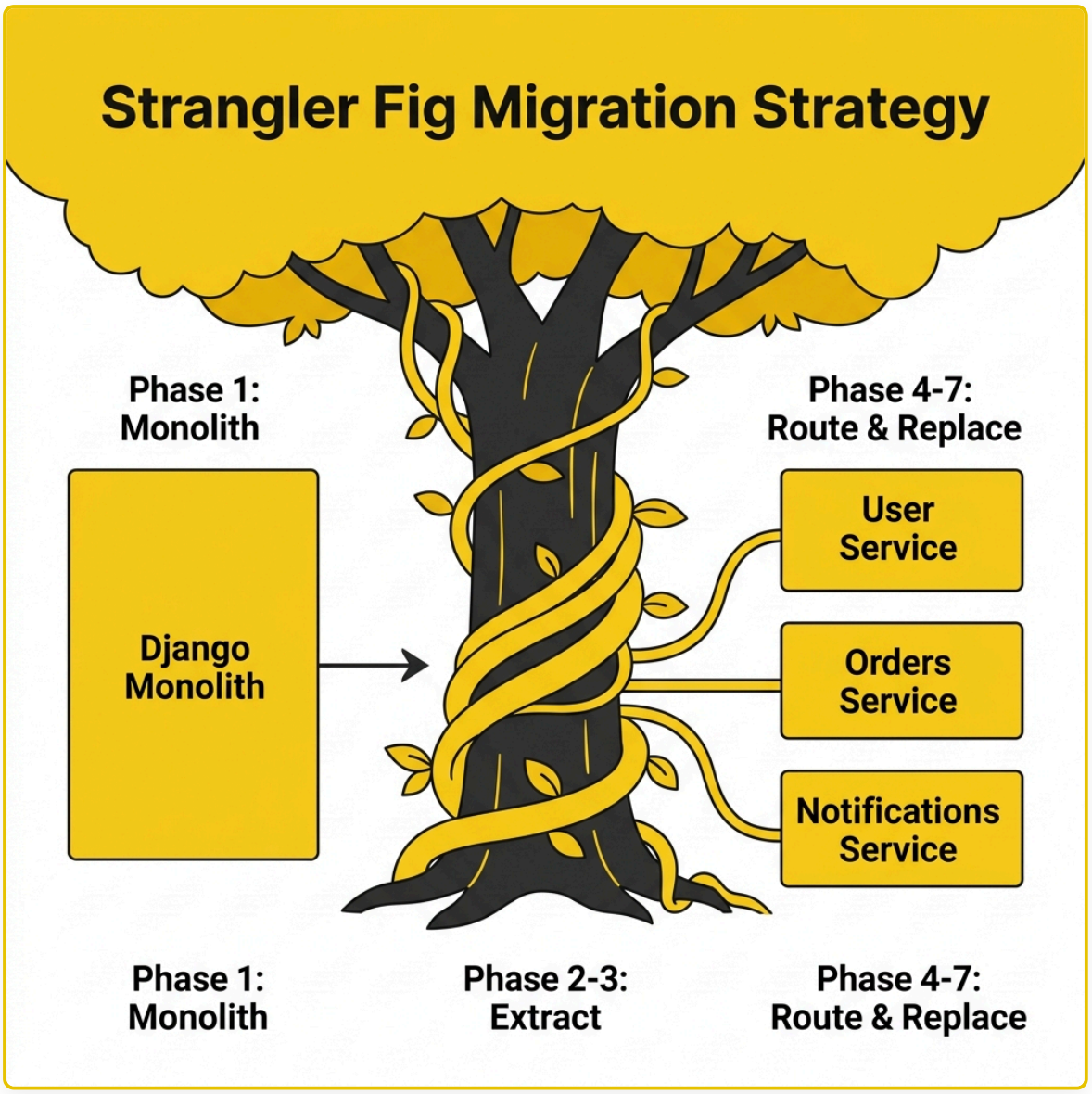
1. Executive Summary

This document walks through a production-grade migration of a Django e-commerce monolith into independent microservices using the **Strangler Fig Pattern**. The migration runs on **AWS EKS** with **Istio** for traffic control, achieving zero downtime through progressive canary releases and instant rollback capability. Every phase keeps the system fully operational — users never notice the transition.



The monolith — which originally handled users, orders, payments, notifications, and administration in a single Django codebase — is incrementally decomposed. Three bounded contexts have been extracted into independent services, reducing the monolith to roughly thirty percent of its original size.

2. The Strangler Fig Pattern

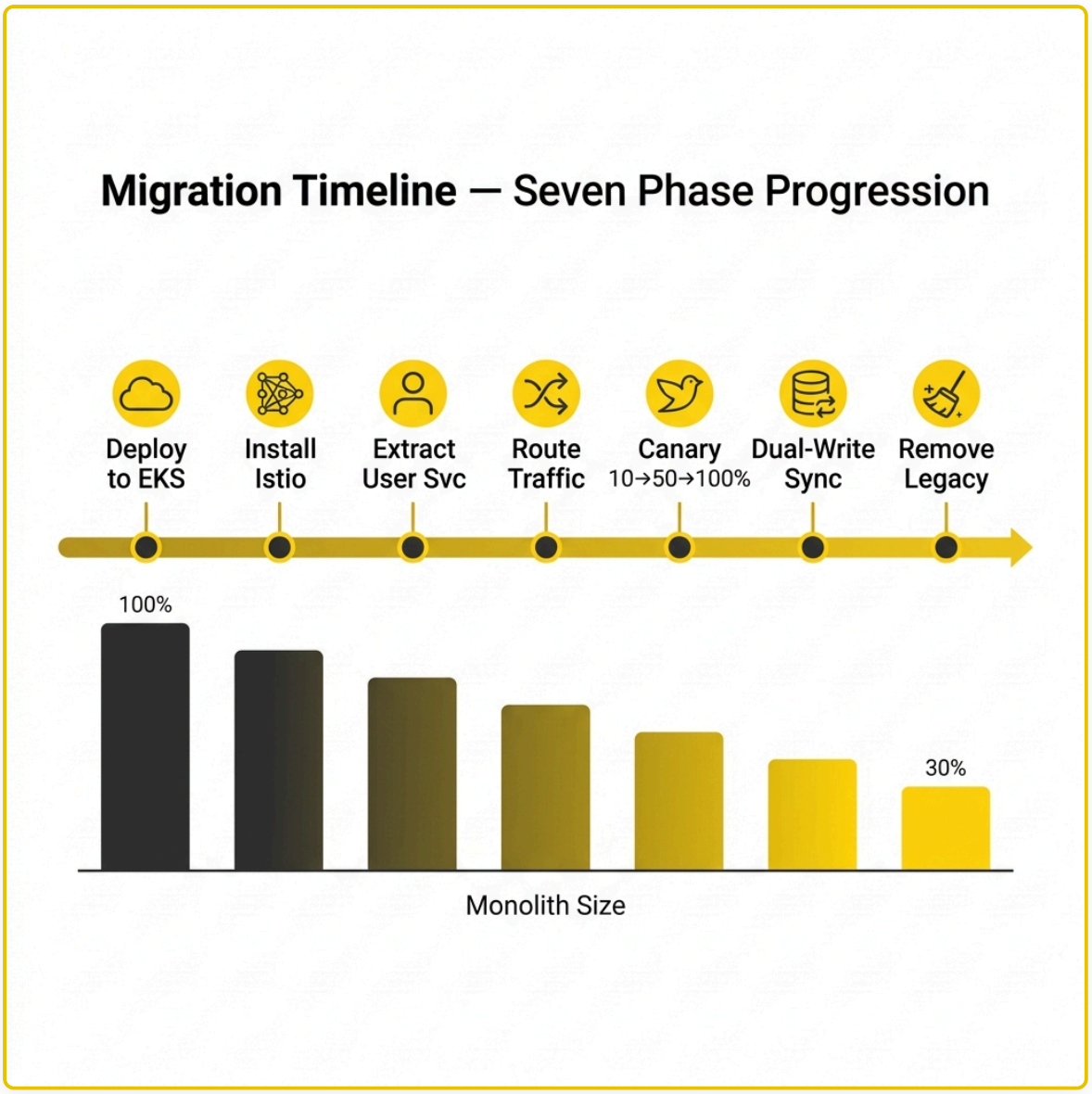


The pattern is named after the strangler fig tree, which grows around a host tree and gradually replaces it. In software, this translates to a simple strategy: place a routing layer in front of the monolith, build new features as microservices, and progressively redirect traffic from old to new. The monolith and microservices coexist throughout — you can stop, pause, or reverse at any point.

Why not a big-bang rewrite? Big-bang rewrites are risky because you discover the new system doesn't work only after committing to the switch. The Strangler Fig pattern eliminates this risk by keeping both systems running and gradually shifting load.

Core mechanism: Istio VirtualService routing rules control which backend handles each incoming request. Combined with a dual-write data synchronization strategy, this ensures both correctness and continuity during the transition.

3. Migration Timeline & Technology Stack



The migration unfolds across seven phases. The monolith shrinks from handling one hundred percent of all functionality down to approximately thirty percent by Phase 7. Each phase is independently reversible.

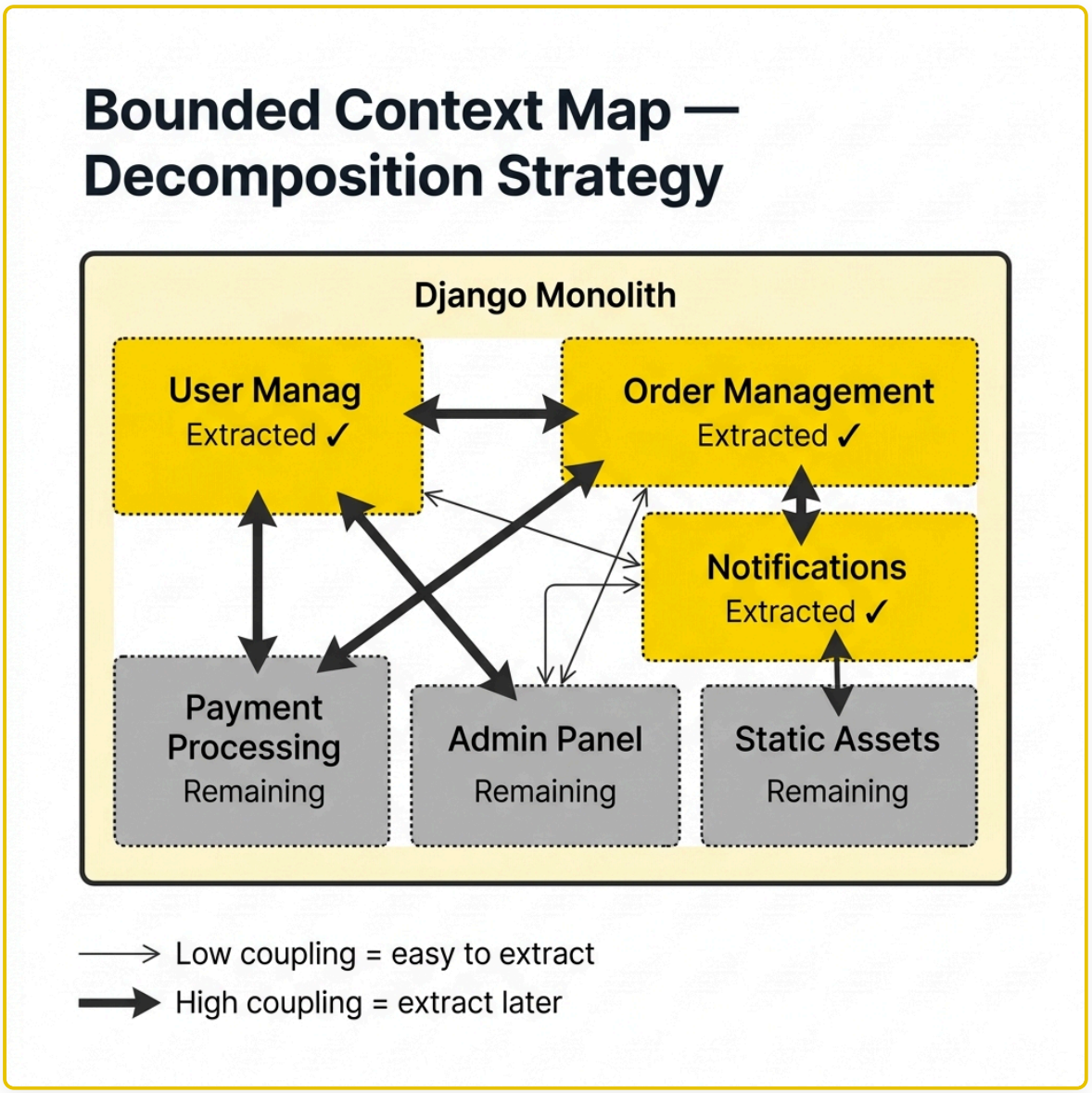
Layer	Technology	Purpose
Infrastructure	AWS EKS, RDS, MSK, ElastiCache	Managed Kubernetes + data services
Service Mesh	Istio + Envoy sidecars	Traffic routing, mTLS, observability
Monolith	Django 4.2 + DRF	Legacy application (shrinking)
Orders Service	FastAPI (Python)	Order management, v2 API
User Service	Flask (Python)	User management CRUD
Notifications	Node.js + Express	Kafka event consumer → emails
Frontend	React (Vite) + Nginx	SPA storefront

CI/CD	GitHub Actions + Helm	Automated build, test, deploy

The polyglot technology choices demonstrate a key microservices advantage: each team picks the best tool for their domain — FastAPI for high-throughput async order processing, Flask for straightforward user CRUD, Node.js for event-driven notification handling.

---

4. Bounded Context Decomposition



Before extracting any service, the monolith must be analyzed to identify **bounded contexts** — self-contained domains with clear boundaries and minimal coupling to other domains.

The diagram above shows the coupling analysis. User Management, Order Management, and Notifications all have low coupling to the rest of the system, making them ideal first candidates. Payment Processing has high coupling with orders (they share a transactional flow), so it is deferred until after the order v1 API is deprecated.

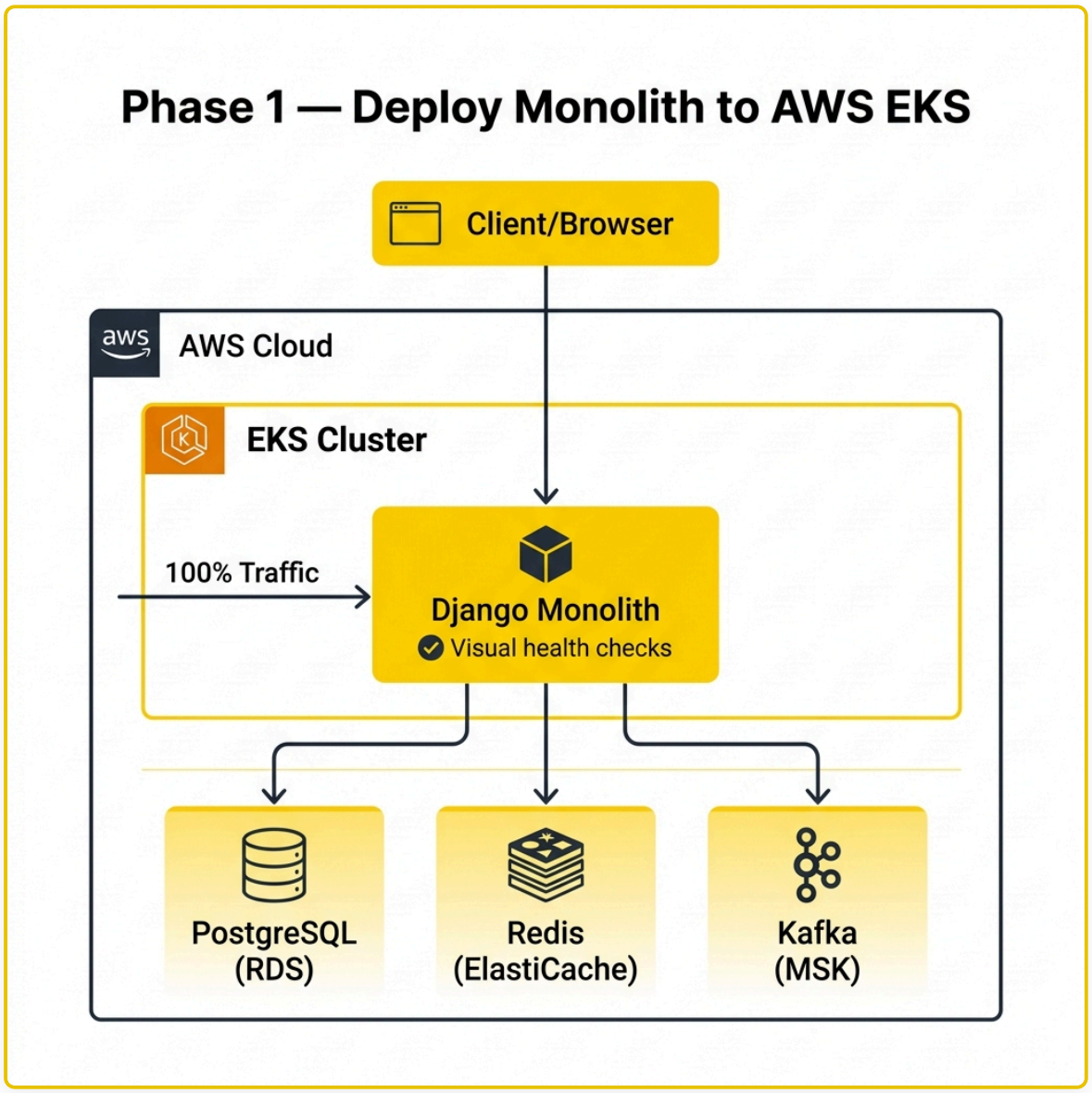
Selection criteria for the first extraction:

Criterion	User Management	Payment Processing
Clear domain boundary	✅ Self-contained CRUD	❌ Tied to order flow
Risk level	✅ Not on revenue path	❌ Directly handles money
Data model complexity	✅ Single table	❌ Multi-table transactions
Inter-service coupling	✅ Minimal	❌ Tightly coupled to orders

User Management was the clear first choice — low risk, clean boundaries, simple data model, and minimal coupling.

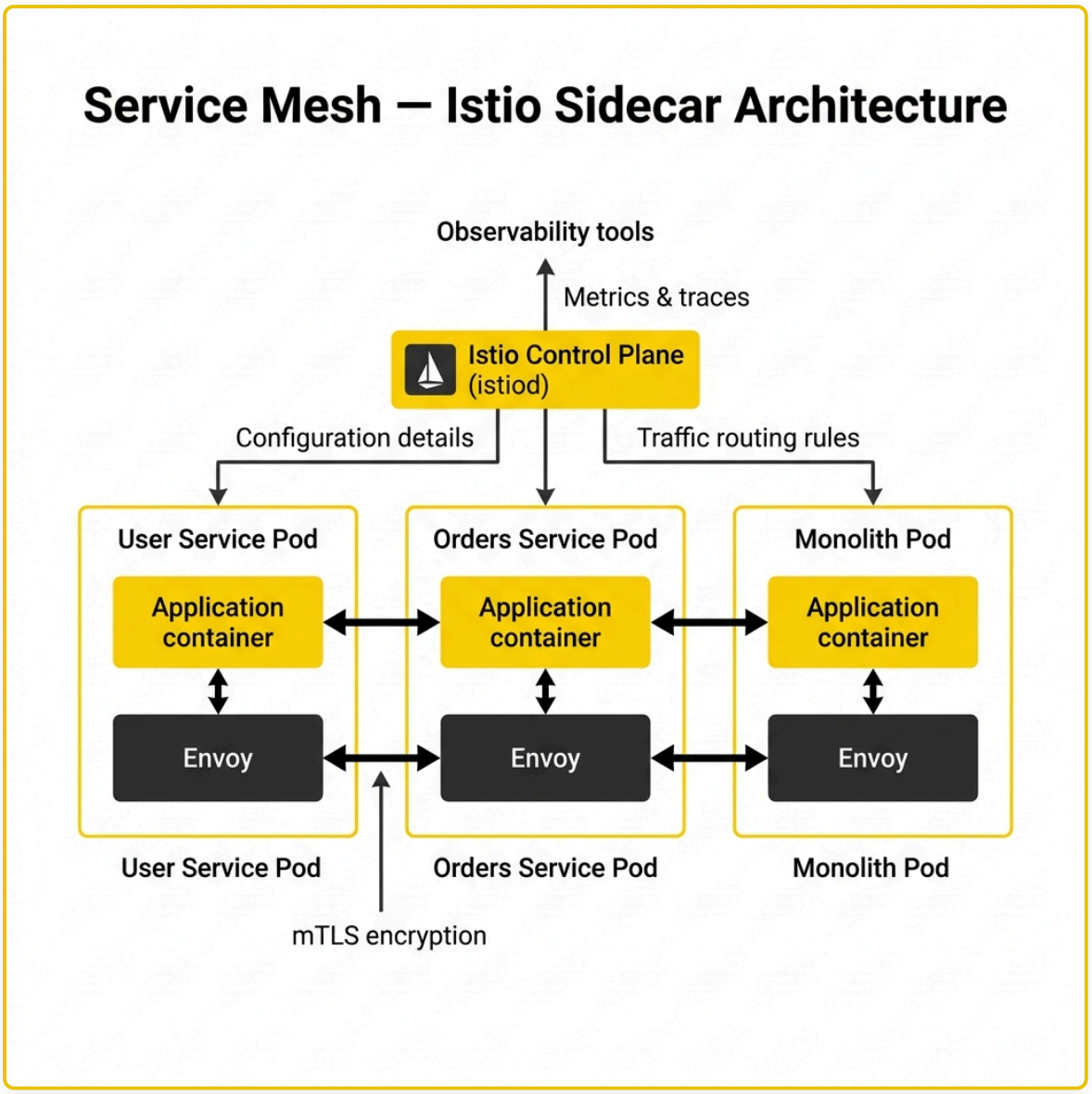
---

5. Phase 1–2: Foundation — EKS Deployment & Istio Mesh



Phase 1 containerizes the Django monolith and deploys it to AWS EKS. The monolith gets health check endpoints for Kubernetes readiness and liveness probes, explicit CPU and memory resource limits, and version labels for Istio routing. All AWS infrastructure — EKS cluster, VPC, RDS, MSK, ElastiCache — is provisioned via Terraform (Infrastructure as Code).

At this stage, the system works identically to before — it's just running on the platform where microservices will eventually join it.

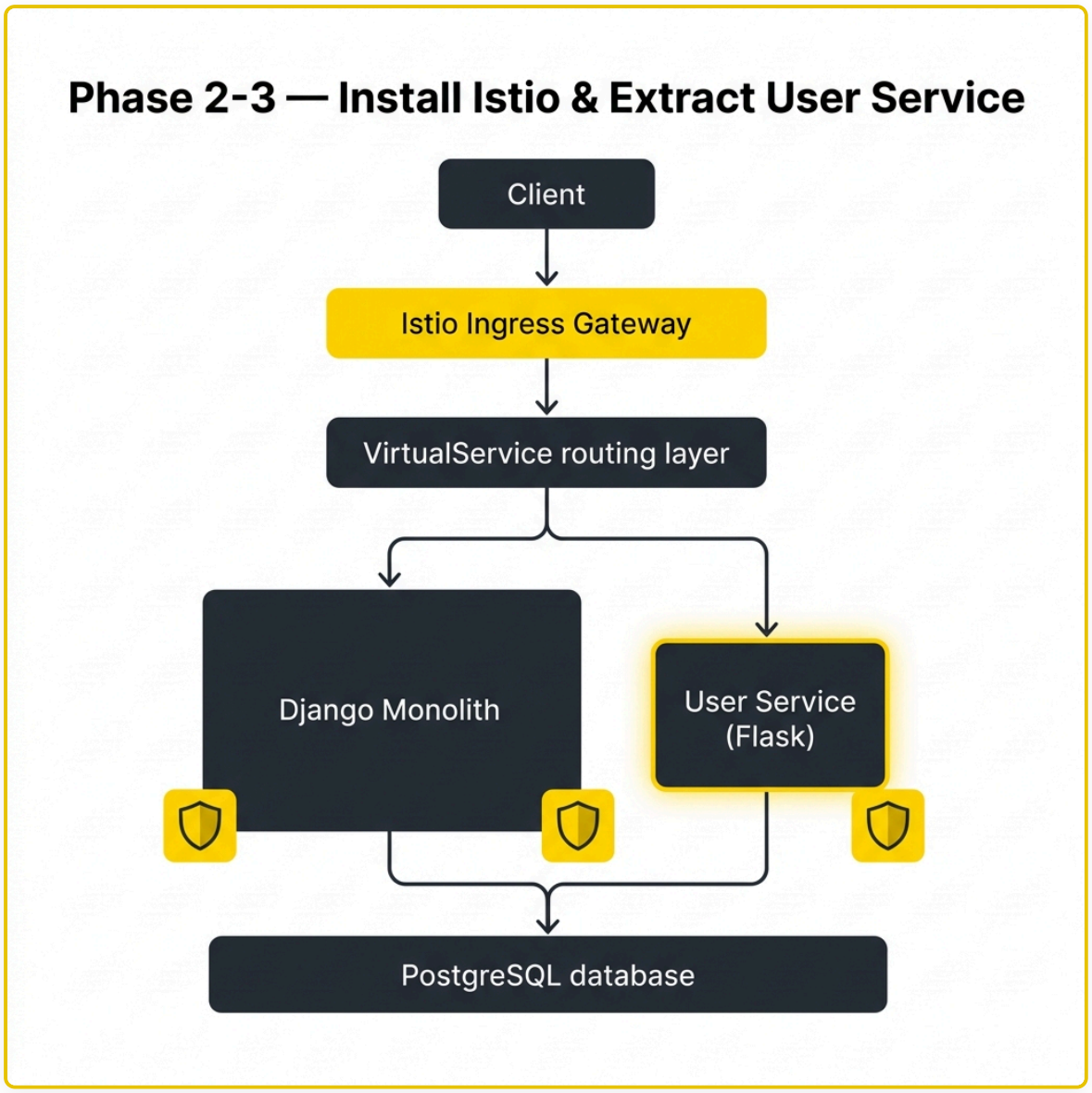


Phase 2 installs the Istio service mesh. Istio injects an Envoy sidecar proxy into every pod, creating a transparent network layer. All traffic between services passes through these proxies, giving Istio control over routing, security (mutual TLS), and telemetry — without any application code changes.

Two critical Istio resources are created: a **Gateway** (the external entry point for traffic) and a **VirtualService** (the routing rules). Initially, the VirtualService routes one hundred percent of traffic to the monolith. The routing layer is in place, but nothing changes for users yet.

Observability tools are also deployed: **Kiali** for mesh topology visualization, **Grafana** for metrics dashboards, and **Jaeger** for distributed request tracing. These are essential for making data-driven routing decisions during later phases.

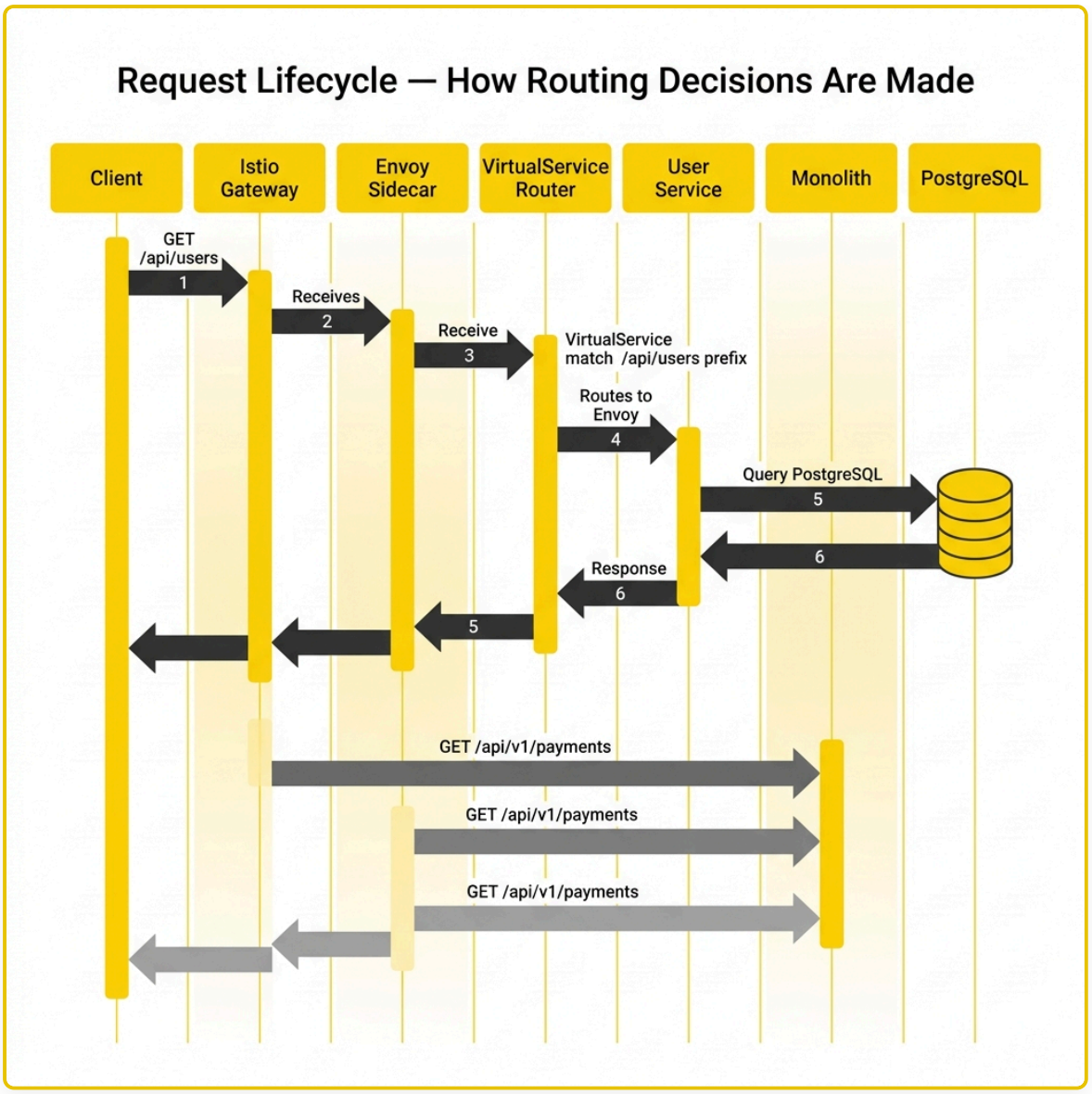
6. Phase 3–4: Extract & Route — The User Service



Phase 3 builds the User Service as a Flask microservice that implements the same API contract as the monolith's user endpoints. It deploys alongside the monolith in the same EKS cluster, with its own pods, resource limits, and health checks. Istio automatically injects sidecar proxies, integrating the service into the mesh.

At this point, both the monolith and the User Service are running — but all traffic still goes to the monolith. The User Service is on standby.

Phase 4 updates the VirtualService routing rules to use **URI prefix matching**: requests to "/api/users" go to the User Service, requests to "/api/v2/orders" go to the Orders Service, and everything else stays with the monolith. This path-based routing is the core Strangler Fig mechanism — specific URL paths are "strangled" away from the monolith while the rest continues as before.

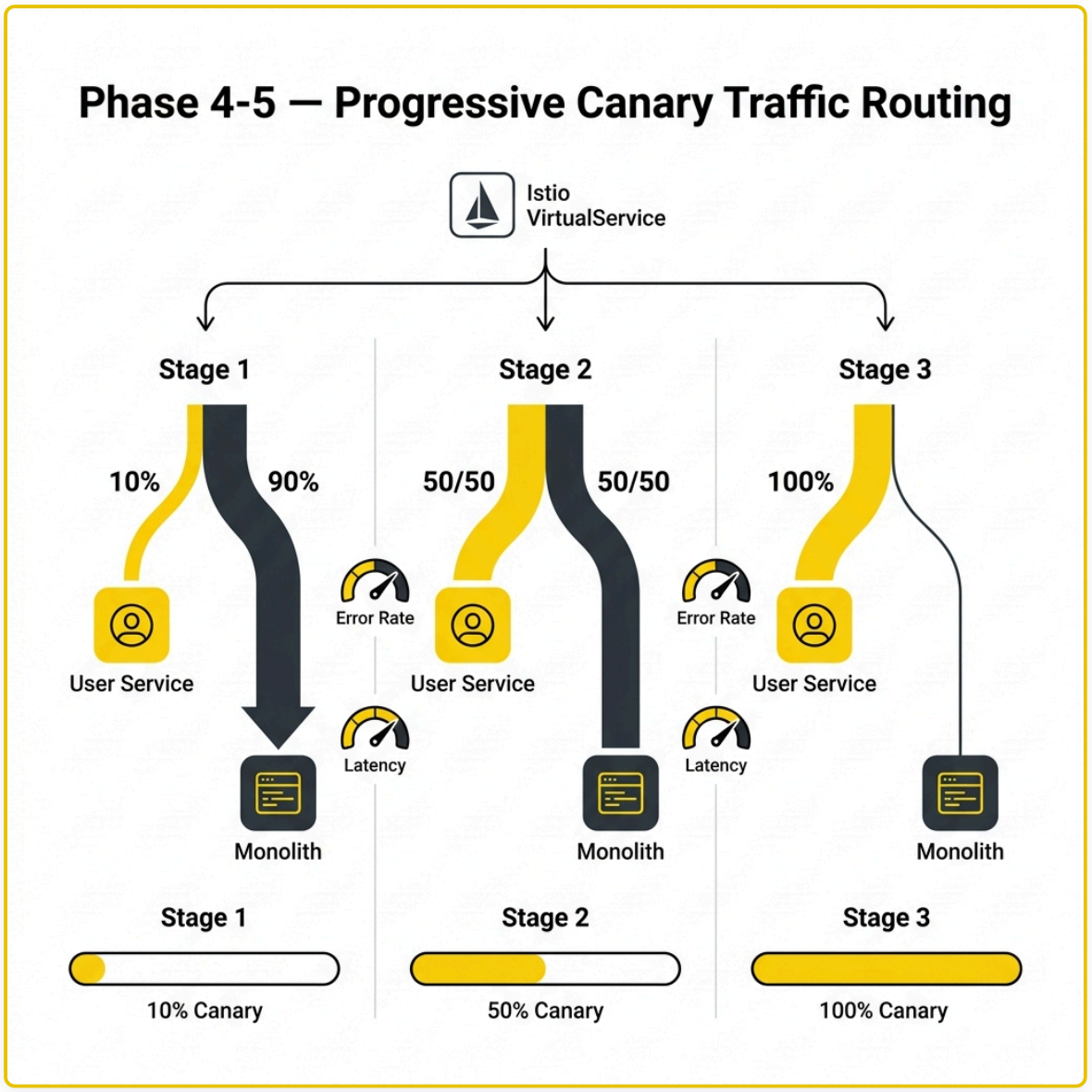


The request lifecycle diagram shows exactly how this works. When a client sends a request to "/api/users", the Istio Gateway receives it, the VirtualService matches the prefix, and traffic is forwarded to the User Service through the Envoy sidecar. A request to "/api/v1/payments" follows the same path but matches the catch-all rule, routing to the monolith instead. From the client's perspective, the same URLs work the same way — the routing is invisible.

Routing order matters: More specific routes (like "/api/users") must be listed before the catch-all. Istio evaluates rules top-to-bottom and applies the first match.

---

7. Phase 5: Canary Traffic Splitting



Rather than switching all user traffic to the new service at once, the cutover happens gradually through a **canary release** — shifting small percentages of traffic and validating performance at each stage.

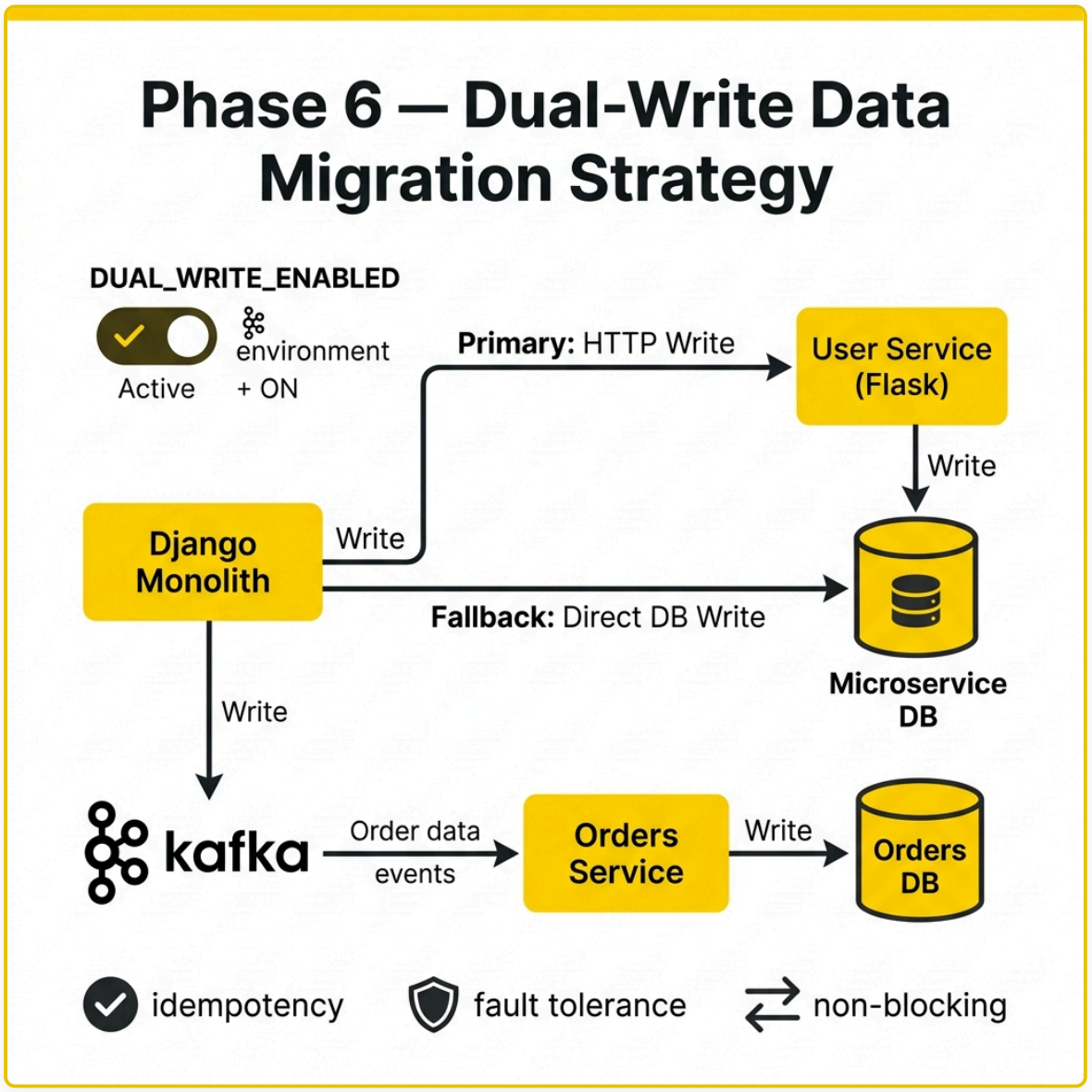
Stage 1 — 10% Canary: One in ten user requests hits the User Service. The team monitors error rates (must stay below one percent), P95 latency (must stay below 500ms), and response correctness. If anything looks wrong, routing reverts instantly.

Stage 2 — 50% Split: Half of user traffic goes to the new service. At this scale, performance comparisons between the monolith and microservice become statistically meaningful.

Stage 3 — 100% Cutover: All user traffic goes to the User Service. The monolith still runs and can accept user requests — it's just not receiving any. Rollback remains a single configuration change away.

An automated rollout script progresses through stages with health validation between each transition. K6 load tests statistically verify the traffic distribution is correct (for example, confirming a 50/50 split is actually producing a 48-52% observed ratio within tolerance).

8. Phase 6: Data Synchronization — Dual-Write Pattern



Data consistency during migration is solved through **dual writes**. While both systems handle traffic, the monolith writes to both its own database and the microservice's database simultaneously.

Two synchronization channels:

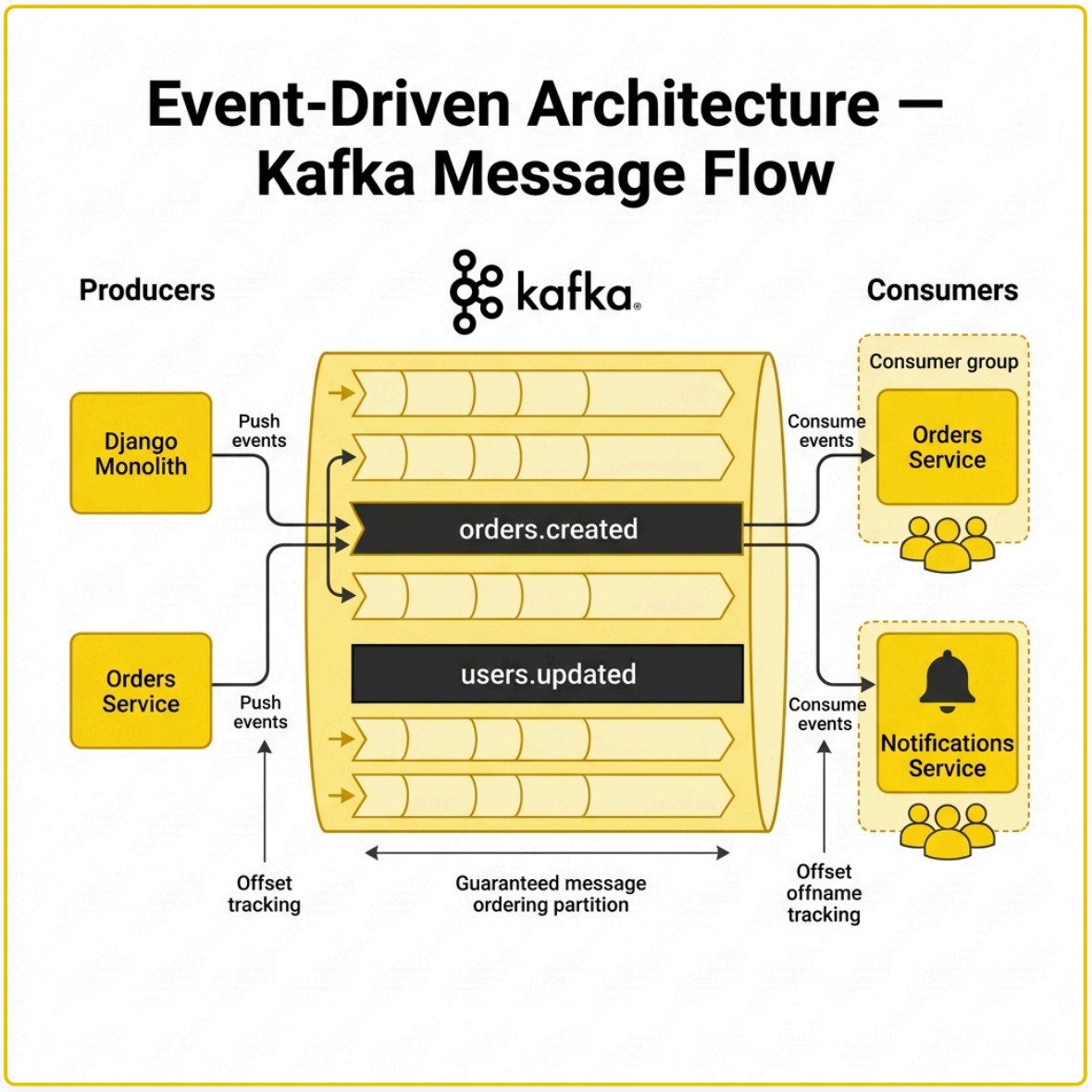
- **Primary — HTTP Write:** The monolith calls the User Service API, which writes to the microservice database. This respects the service's business logic and validation.
- **Fallback — Direct DB Write:** If the HTTP call fails (service down, timeout), the monolith writes directly to the microservice's database table. This ensures consistency even during outages.

Critical design properties:

Property	How It Works
Environment-controlled	Toggle via DUAL_WRITE_ENABLED without redeployment

Idempotent	Duplicate writes are silently ignored (ON CONFLICT DO NOTHING)
Fault-tolerant	HTTP failure triggers DB fallback automatically
Non-blocking	Failures are logged, never block the primary operation

Event-Driven Architecture — Kafka



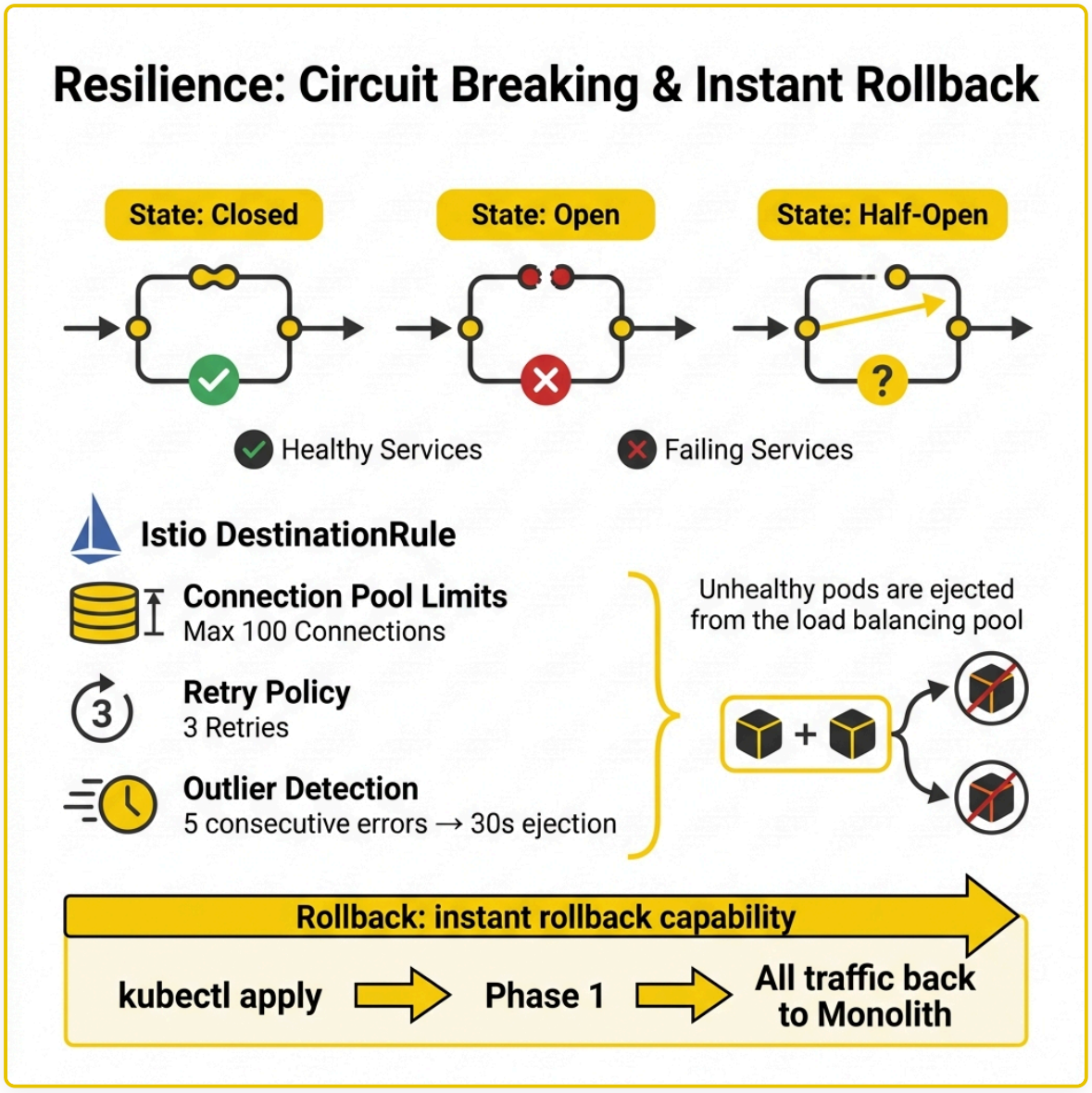
For order data, a different pattern is used: **Kafka event streaming**. The monolith publishes "orders.created" events to Kafka. The Orders Service consumes these events and writes to its own database, providing eventual consistency with guaranteed ordering.

The Notifications Service also consumes from the same Kafka topic — demonstrating the **fan-out pattern**, where a single event triggers independent actions in multiple consumers. When an order is created, the Orders Service records it while the Notifications Service sends a customer email — neither service knows about the other.

This decoupled, event-driven architecture contrasts with the dual-write's synchronous approach: Kafka provides better scalability and looser coupling, while dual-write provides stronger consistency guarantees.

The choice depends on the domain's requirements.

9. Resilience — Circuit Breaking & Rollback



Circuit Breaking

Istio implements the **circuit breaker pattern** through DestinationRules to prevent cascading failures:

- **Closed (Normal):** Traffic flows normally. Errors are counted.
- **Open (Failure Detected):** After five consecutive 5xx errors, traffic is blocked for thirty seconds. The failing service isn't overwhelmed.
- **Half-Open (Recovery Test):** A few requests are allowed through. If they succeed, the circuit closes. If they fail, it opens again.

Additional traffic policies include connection pool limits (max 100 concurrent connections), automatic retries (up to 3 attempts with backoff), and outlier detection (unhealthy pods are ejected from the load balancing

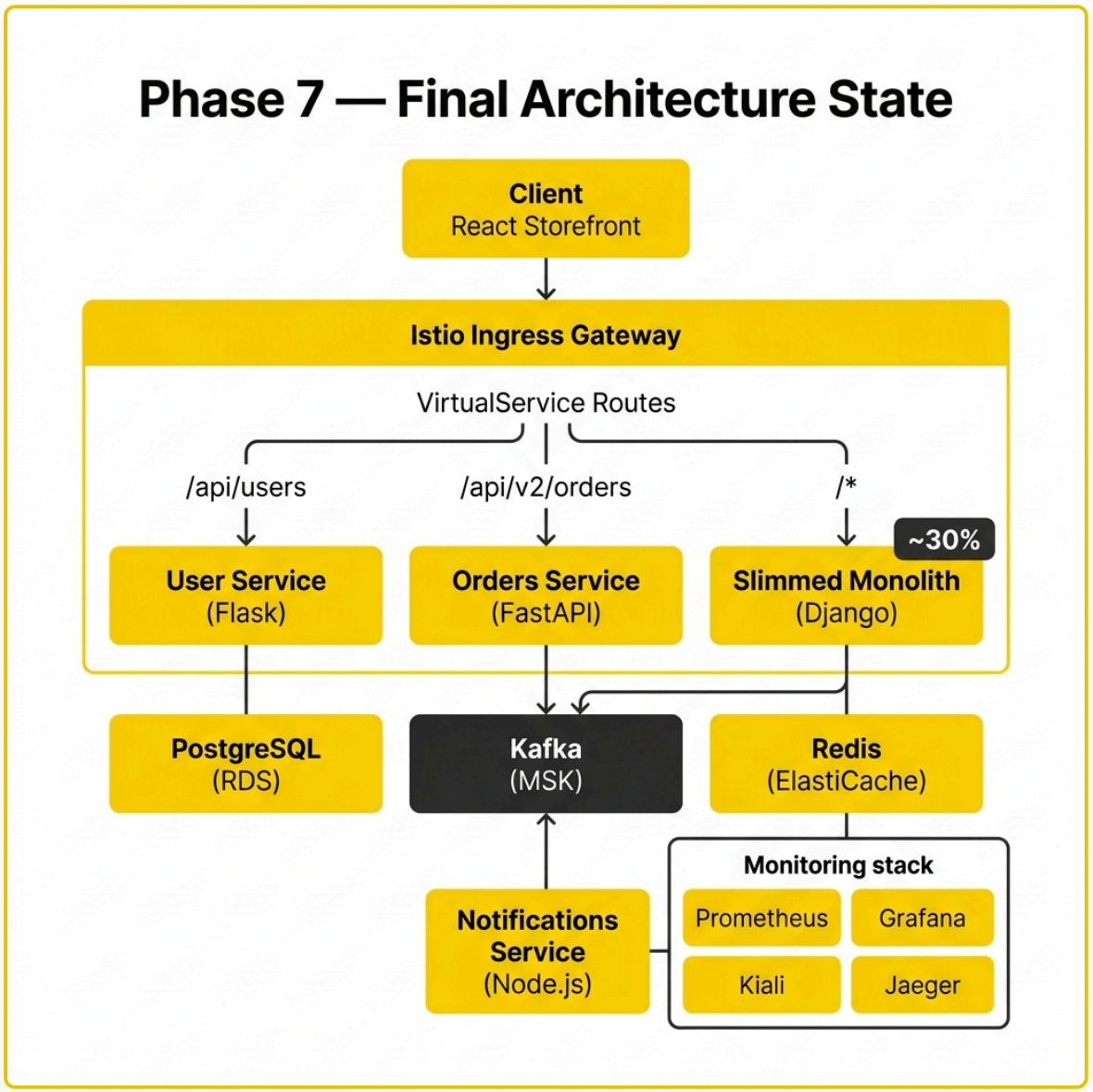
pool for thirty seconds, up to fifty percent of pods).

Instant Rollback

The most important safety mechanism: at any point during the migration, a single configuration change routes one hundred percent of traffic back to the monolith. No pods restart, no code redeploys, no data migration. The monolith has been running the entire time, ready to accept all traffic immediately. This rollback capability is never sacrificed until the migration is fully validated and stable.

---

10. Phase 7: Final Architecture

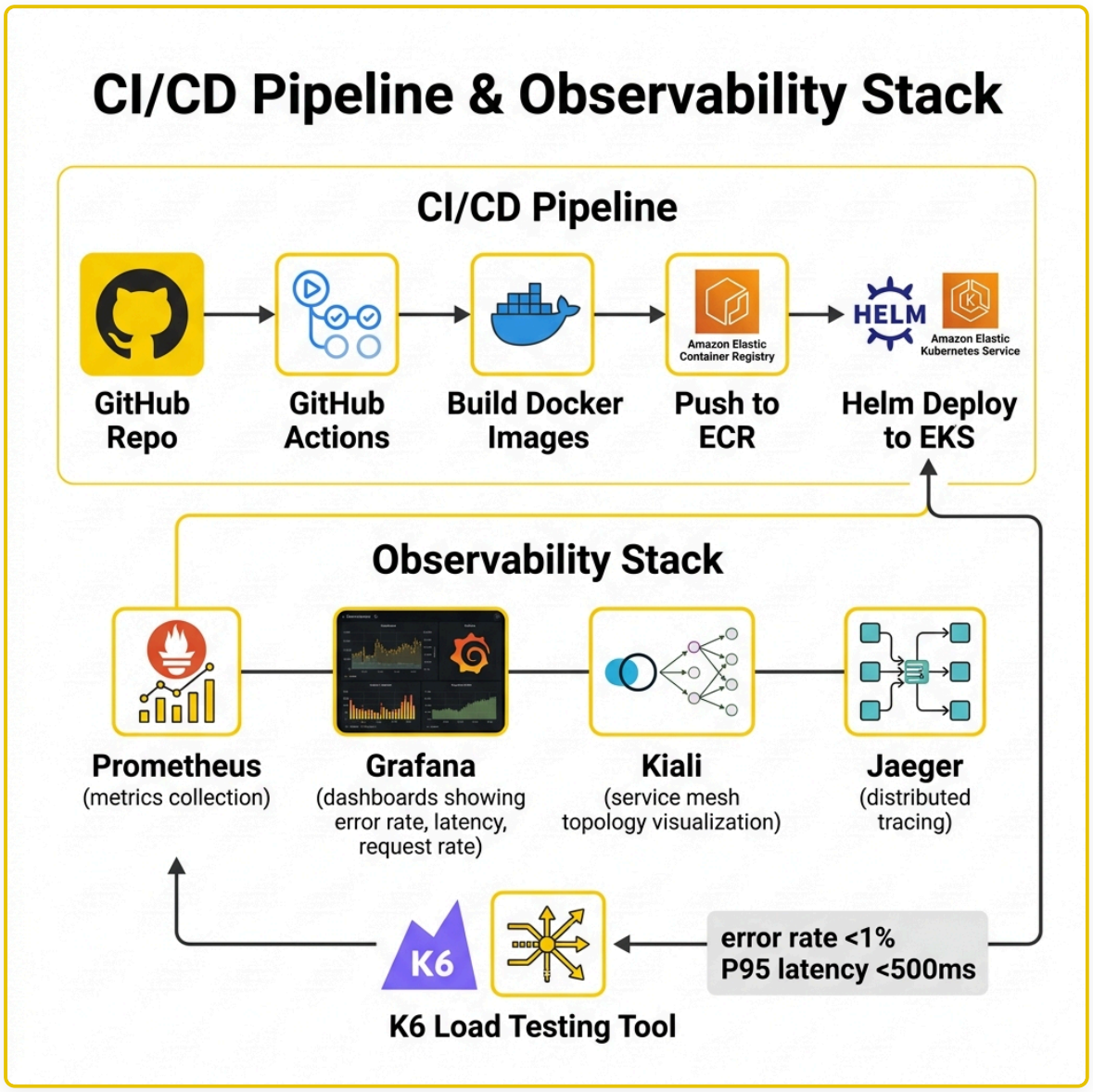


In the final state, the User Service handles "/api/users" (100%), the Orders Service handles "/api/v2/orders" (100%), and the slimmed monolith handles everything remaining — payments, admin, and static assets (~30% of original).

The cleanup process follows a strict sequence: verify zero monolith traffic on extracted endpoints via Istio telemetry → disable dual-writes → remove legacy code from the monolith → rebuild and redeploy the slimmed monolith → apply final routing configuration.

---

11. CI/CD & Observability



CI/CD Pipeline: GitHub repo → GitHub Actions (build, test, lint) → Docker images → Amazon ECR → Helm deploy to EKS. Pre-commit hooks enforce code quality with Black formatting and security scanning.

Observability during migration — key metrics to watch:

Metric	Tool	Target
Error rate (5xx)	Prometheus / Grafana	< 1%
P95 latency	Prometheus / Grafana	< 500ms

Request rate	Prometheus / Grafana	Stable ± 5%
Kafka consumer lag	MSK metrics	< 100 messages
Pod restarts	Kubernetes	0
Traffic split accuracy	K6 canary tests	Within ± 10% tolerance

Kiali provides real-time mesh topology — visually confirming traffic flows during canary releases. **Jaeger** provides distributed tracing to pinpoint exactly where latency is introduced when comparing monolith vs. microservice performance.

---

12. Migration Status & Lessons Learned

Current Status

Bounded Context	Service	Status	Data Strategy
Order Management	Orders Svc (FastAPI)	✅ Extracted	Kafka event streaming
Notifications	Notifications Svc (Node.js)	✅ Extracted	Kafka consumer (event-driven)
User Management	User Svc (Flask)	✅ Extracted	Dual-write (HTTP + DB fallback)
Payment Processing	—	🔲 Remaining	Pending order v1 deprecation
Admin Panel	—	🔲 Remaining	Low priority, may stay in monolith
Static Assets	—	🔲 Remaining	Recommend moving to CDN

Key Lessons

Start with the easiest extraction. Low-risk, well-bounded contexts let the team learn patterns and tooling before tackling critical business domains.

Observability before routing changes. Baseline metrics from the monolith are needed to compare against microservice performance. Without data, routing decisions are based on hope.

Dual-write is temporary. It ensures data consistency during transition but adds complexity. Turn it off the moment the microservice is the sole source of truth.

Always keep the rollback path open. The ability to instantly revert all traffic to the monolith is the single most valuable safety feature. Never compromise it until migration is fully validated.

Canary releases prevent surprises. Progressive traffic shifting (10% → 50% → 100%) with health validation catches issues early, when they affect only a small fraction of users.
